

Investigating the Use of Productive Failure as a Design Paradigm for Learning Introductory Python Programming

Hussel Suriyaarachchi
Augmented Human Lab
National University of Singapore
Singapore
hussel@ahlab.org

Paul Denny
School of Computer Science
University of Auckland
Auckland, New Zealand
paul@cs.auckland.ac.nz

Suranga Nanayakkara
Augmented Human Lab
National University of Singapore
Singapore
suranga@ahlab.org

Abstract

Productive Failure (PF) is a learning approach where students initially tackle novel problems targeting concepts they have not yet learned, followed by a consolidation phase where these concepts are taught. Recent application in STEM disciplines suggests that PF can help learners develop more robust conceptual knowledge. However, empirical validation of PF for programming education remains under-explored. In this paper, we investigate the use of PF to teach Python lists to undergraduate students with limited prior programming experience. We designed a novel PF-based learning activity that incorporated the unobtrusive collection of real-time heart-rate data from consumer-grade wearable sensors. This sensor data was used both to make the learning activity engaging and to infer cognitive load. We evaluated our approach with 20 participants, half of whom were taught Python concepts using Direct Instruction (DI), and the other half with PF. We found that although there was no difference in initial learning outcomes between the groups, students who followed the PF approach showed better knowledge retention and performance on delayed but similar tasks. In addition, physiological measurements indicated that these students also exhibited a larger decrease in cognitive load during their tasks after instruction. Our findings suggest that PF-based approaches may lead to more robust learning, and that future work should investigate similar activities at scale across a range of concepts.

CCS Concepts

• **Social and professional topics** → *Computing education*.

Keywords

Productive Failure, CS1/CS2, Physical Computing

ACM Reference Format:

Hussel Suriyaarachchi, Paul Denny, and Suranga Nanayakkara. 2025. Investigating the Use of Productive Failure as a Design Paradigm for Learning Introductory Python Programming. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE TS 2025)*, February 26-March 1, 2025, Pittsburgh, PA, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3641554.3701911>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGCSE TS 2025, February 26-March 1, 2025, Pittsburgh, PA, USA
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0531-1/25/02
<https://doi.org/10.1145/3641554.3701911>

1 Introduction

Productive Failure (PF) is an instructional approach where students initially engage with problems targeting concepts they have not yet learned, followed by a phase of consolidation where these concepts are taught [11, 13]. The rationale behind PF is that initial struggle and failure can activate prior knowledge and create a fertile ground for subsequent learning, leading to deeper understanding and better retention of concepts [12].

Since its introduction, PF has gained recognition among researchers and educators for its success in STEM disciplines, most popularly mathematics and physics, where it has been shown to promote more robust learning outcomes compared to traditional instructional methods [13, 26, 37]. Despite these successes in other fields, there has been little empirical work exploring the application of PF in computing or programming education in comparison to more traditional Direct Instruction (DI) approaches. Recent work indicates that PF may be a promising approach for computing education [23, 35], but there is a need for more work on designing appropriate tasks and for objective data on its effects.

To address this gap, in this paper, we describe our design and development of a novel PF-based activity targeting lists in Python. This activity was informed by an iterative, student-centred design approach involving pilot studies with participants of varied programming proficiencies. We incorporate real-time data collection during the learning activity, utilising consumer-grade wearable sensors to measure physiological data. This data was used by students as part of the programming task, and it also allowed us to infer cognitive load.

We conducted a controlled study involving 20 participants to evaluate the impact of our PF-based activity on learning outcomes and cognitive load, in comparison to traditional Direct Instruction (DI), measuring performance on a two-week delayed post-test task. Our research is guided by the following questions:

- RQ1.** How does the choice of pedagogical design impact initial task performance and subsequent success on a delayed task?
- RQ2.** To what extent do physiological measures of cognitive load change after students receive programming instruction, and does PF influence these changes?
- RQ3.** What are students' preferences and perceptions regarding PF and DI?

Our findings show that students performed similarly on programming tasks immediately after instruction, regardless of the pedagogical strategy. However, those using the PF approach performed better on the same tasks two weeks later. We also explore the implications for reducing cognitive load and its link to improved performance.

2 Related work

Knowledge of computing, and specifically programming, has become a core skill that shapes how students think and tackle problems across various subject domains [6]. There has been considerable research interest in exploring and developing effective pedagogical approaches for teaching programming [5].

2.1 Productive Failure in Computing Education

Productive Failure (PF) [11] is an established instructional framework that lends itself to constructionist learning [8] in which students engage in a Problem-Solving followed-by-instruction (PS-I) [15, 24] learning design to construct new knowledge. In recent years, PF has begun to appear in computing education, with emerging use in introductory CS1/CS2 undergraduate programs.

Preliminary work by Thorgeirsson et al. [35] explored the use of PF to introduce programming concepts such as sorting algorithms, graph theory, and cluster assignment. Their failure-driven learning experience was facilitated through prompts that deliberately nudged students towards suboptimal solutions within a visual programming environment named *Algot* [36]. Similarly, Savelson and Muldner [22] delivered their PF strategy on sorting using a custom interface of the popular block-based programming platform *Snap!*¹. These studies showed favourable effects in improving students' constructive reasoning and curiosity compared to DI approaches while maintaining similar frustration levels.

Although this recent work indicated positive learning behaviours, there was inconclusive evidence on the impact of PF on learner performance and conceptual understanding. Steinhorst et al. [29, 30] also noted these concerns in their PF interventions on computing education. Their results indicated that the learning advantages of PF over DI observed in other fields, with respect to conceptual knowledge and task performance, could not be replicated for computing concepts on operating systems. Thus, considering the infancy of PF in computing education and its known potential for robust learning, there is a need for more empirical work to reliably determine its efficacy.

2.2 Understanding Learning

Understanding what it means to learn is a particularly critical but challenging task in applied educational settings [2]. The assessment of learning and its outcomes is typically informed by the achievement of curricular goals and skills [28]. Instructors rely on diagnostic and summative strategies such as self-reported measures and standardised tests to gauge learning performance [18, 19]. While these conventional academic metrics may be functional, they fail to accurately represent learning processes [20]. Thus, researchers need to confront the task of providing more reliable and objective measures of learning [1].

With learning being a complex and multi-faceted process embodying both the body and mind, there has been much interest in exploring its relationship to various cognitive and affective states [4, 27, 33]. Variations observed in students' physiology are increasingly recognised as effective indicators of factors that underpin learning, such as cognitive load [10, 17]. For example, heart rate variability (HRV) is one such physiological measure that is closely

linked to cognitive load [7]. Prior work exploring cognitive load in computing and programming tasks found that experiencing a reduced cognitive load tends to improve student learning [21, 32].

In light of the recent wave of consumer-friendly physiological sensors and smart wearables, the possibility of seamlessly monitoring cognitive and physiological functions in the classroom lies within reach. However, introducing tools that are not directly aligned with learning activities may be distracting for students and disrupt authentic learning processes [31].

In this work, we explore the use of Productive Failure in computing education and investigate its potential in an introductory Python undergraduate course. We incorporate wearable consumer sensors as essential parts of learning activities to make their presence intuitive to students and to enable the unobtrusive collection of physiological data on authentic learning processes.

3 PF Design & Development

To explore our research questions, we developed a Python programming task focused on lists. The choice of concept was motivated by ACM's curriculum guidelines for undergraduate computer science education [9], which identify lists as a popular CS1 construct. Lists are also an ideal topic for designing problem-solving contexts that embody Kapur et al.'s recommendations for PF learning experiences, such as accommodating various solution approaches [13].

3.1 Programming Task

For our study, we created an open-ended challenge that targeted Python lists and involved storing data from a heart-rate sensor.

3.1.1 Python List Task. The task on lists was developed as a simplified sliding-window problem on a live stream of heart-rate data and is accessible online². Students are required to explore using their existing knowledge of data structures and variables to maintain a record of the ten most recent heart rate readings and ensure that this data stored is always up-to-date. While the canonical solution involves using a list to 'append' and 'pop' data based on its length, our task design also supports the generation of suboptimal representations and solution methods using constructs such as strings and temporary variables.

3.1.2 Pilot Evaluation. After initial development of the task, and prior to our main evaluation, we conducted a pilot study to ensure the task was reliably understood and successfully promoted

²<https://bit.ly/pf-list-task>

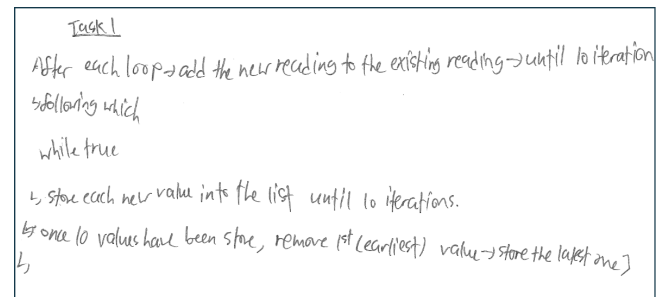


Figure 1: A handwritten solution collected during the pilot evaluation illustrating exploration of the problem.

¹<https://snap.berkeley.edu/>

problem-solving behaviours representative of a PF learning exercise. This pilot was structured as a think-aloud study with 11 students from a first-year undergraduate Python course. Students attempted the programming task on pen and paper, and were encouraged to describe their problem-solving thought processes as pseudocode if they struggled to generate a Python-based solution. Figure 1 illustrates one example of a student’s exploration, highlighting the opportunity for multiple solution pathways using representations that do not involve the relevant canonical method.

3.2 Python Library

We developed an open-source Python library³ as a software package that can be installed using Python’s *pip* package management system. We use a consumer-grade wearable heart-rate sensor from Polar to facilitate the use of sensor data in our implementation. The Polar Verity Sense⁴ wristband is an optical heart rate sensor that captures physiological photoplethysmography (PPG) data, which can be used to measure features of Heart Rate Variability (HRV). We employ a Bluetooth Low Energy (BLE) communication protocol for wireless data transmission between the wearable sensor and our library. All operations for device connectivity and data processing are handled automatically by our library. For example, to access heart-rate data, users simply call a function named “get_latest_heart_rate()” in their program. Thus, our library offers seamless plug-and-play use of sensor data, eliminating complicated technical configurations to simplify its use in a classroom context.

4 Evaluation

4.1 Participants

Our evaluation involved 20 students (9 male, 11 female) who were enrolled in an undergraduate CS1 Python programming course. At the time of our evaluation, students were in their third week of their semester and had not yet been introduced to lists.

4.2 Procedure

We designed a controlled study with a subsequent short-term follow-up phase. Both the controlled study and the follow-up were conducted as individual sessions for each student and scheduled two weeks apart. Figure 2 provides an overview of the evaluation protocol we followed, which we expand on below.

4.2.1 Initial Session (90 minutes). The initial session composed of a short introduction (15 minutes), a lesson on Python lists (45 minutes), and a programming activity (30 minutes). We randomly allocated half of the students to the experimental condition, where PF was used as the instructional strategy for the lesson on lists. The remaining students served as the control condition, learning about lists through traditional DI.

The introduction was used to brief students on the agenda of the session and to establish a baseline period of neutral cognitive load (see Section 4.3.2 for the calculation of cognitive load from PPG data). Students were asked to wear the heart-rate sensor on their forearm and relax for 5 minutes to collect the PPG data to determine this baseline value. The sensor was worn for the remainder of the

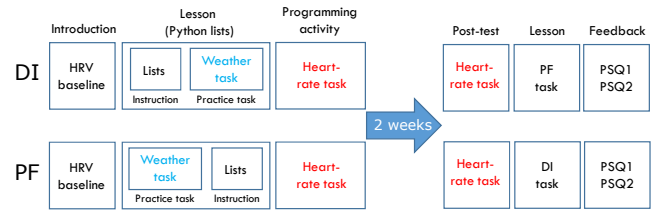


Figure 2: Overview of the study procedure for the evaluation of our PF learning experience (against DI as a control).

study, enabling the use of heart-rate data in the programming tasks and the continued collection of PPG data to infer cognitive load.

The lesson on lists involved a 30-minute practice task and a 15-minute instructional unit. We developed the practice task for this lesson by creating an isomorphic version of our programming task on lists described in Section 3.1. The isomorphic exercise was based on an identical sliding-window problem but relied on a temperature data stream from a weather station instead of live heart-rate data (and required tracking the most recent seven readings). In the experimental PF condition, students initially attempted the practice task, followed by a consolidation phase where shortcomings of their solutions were discussed and list concepts were explained. Conversely, the control DI condition began with an introduction to lists, after which students were given the same practice task. The session concluded with all students engaging in our programming task using heart-rate data where they applied their new knowledge of lists.

4.2.2 Follow-up Session (45 minutes). Students returned two weeks later for their follow-up session, which included a post-test (10 minutes), a short lesson on a new concept (30 minutes), and a feedback session (5 minutes). We asked students to wear the sensor and engage in a post-test activity where they re-attempted the heart-rate-based programming task from their first session. Our focus in this segment was to gauge students’ performance in delayed tasks after receiving instruction.

Following the post-test, students participated in a short lesson on Python dictionaries, which was taught using the instructional condition they had not received in their initial session. Thus, students who experienced the PF condition in their first session received DI in their second session and vice versa. The motivation for this approach was to provide all students the opportunity to express their perceptions of the learning experiences provided by both DI and PF. At the end of the lesson, we administered a questionnaire via Google Forms containing two open-ended questions:

- PSQ1.** Please reflect on your experience with the study and share any thoughts you have about learning Python lists and dictionaries.
- PSQ2.** Each of the two sessions that you attended used a different strategy to teach a programming concept (list or dictionary). Which of the sessions worked better for you and why?

These questions aimed to gather insights into students’ perceptions and experiences of learning Python programming with DI and PF.

³<https://github.com/augmented-human-lab/polar-ble-python>

⁴<https://www.polar.com/en/products/accessories/polar-verity-sense>

Group	Initial Success		Post-test Success
	Weather	Heart-Rate	Heart-Rate
DI ($n = 9$)	8/9	8/9	6/9
PF ($n = 7$)	2/7	7/7	7/7

Table 1: Summary of students’ success in their initial tasks and the post-test task 2 weeks-later.

4.3 Data Analysis

We received completed questionnaires from 16 (out of 20) students who participated in our study and analysed their data.

4.3.1 Programming performance. The two programs (weather station and heart-rate) developed by each student during the initial session (Python lists) were used to explore how well they constructed solutions. We divided the collected data into two groups: control (DI; $n = 9$) and experimental (PF; $n = 7$). Unit testing was performed to determine if these programs produced the correct outputs for a given set of input data streams. In addition, we conducted a static analysis of the students’ code to understand how their solution implementations compared to the canonical method and to classify any errors.

Similar measures were derived for the post-test programming task (heart-rate) in the follow-up session. This provided paired performance data on the heart-rate programming task for each student. We assessed learning outcomes by evaluating the changes in students’ performance on this programming task that occurred over the two-week interval between sessions.

4.3.2 Cognitive Load. We established a data processing pipeline using HeartPy [38] to infer cognitive load from the PPG data collected during the first session. PPG is a physiological signal representing blood volume pulse that can be used to compute Heart Rate Variability (HRV) by analysing the time intervals between successive peaks in its waveform. HRV is a reliable marker of cognitive load [7], with a decrease in HRV indicating an increase in cognitive load [34]. We calculate the root mean square of successive differences between normal heartbeats (RMSSD), which is a widely used time-domain HRV metric [25], to extract cognitive load measures in our analysis. Changes in students’ RMSSD values relative to their baseline at the start of the first session were used to infer the effects of DI and PF on cognitive load.

4.3.3 Qualitative Analysis. Thematic analysis was performed on the free-form responses to understand students’ experiences and preferences of the learning strategies introduced. Following the guidelines by Braun and Clarke [3], we tagged responses to *PSQ1* and *PSQ2* and synthesised them into high-level themes.

5 Results & Discussion

5.1 Student Performance

A summary of students’ performance in their respective tasks on Python lists during the initial session and the follow-up session are presented in Table 1.

5.1.1 Learning with Direct Instruction. Students in the control group ($n = 9$) demonstrated high proficiency using lists during the practice task on weather station data, which they attempted immediately after receiving instruction. Eight of the nine programs analysed

executed successfully against our test cases and displayed optimal use of the list data structure.

When developing the solution for the weather station problem, students who received direct instruction adopted a concept-first approach, using their newly acquired knowledge of lists to guide the overall structure of their program. The example program shown in Listing 1 illustrates the standard solution procedure followed. Students used a conditional code block based on the size of the list to determine when data should be added or removed. In this instance, the student applied the `len` function to assess the length of the list, followed by the `append` and `pop` methods to add or remove values from the list, respectively. All accepted solutions were constructed using appropriate list methods taught during the learning phase. The `append` method was a popular choice for adding data to a list and was seen used across all programs. In contrast, we observed that a few students opted for alternatives to the ‘`pop`’ method, such as `del`, `remove` and slicing, for deleting list items. Providing students with an initial conceptual understanding of lists prior to engaging in hands-on programming may have motivated a deeper exploration of list operations beyond those presented in our starter Python tutorial. Naturally, the results from the heart-rate-based programming task that followed shortly afterwards revealed identical performance, with the same eight students achieving correct solutions.

```

1 from weather_station import *
2
3 output_list = []
4 while NEW_DAY_AVAILABLE: # This simulates a new day
5     temperature_today = get_daily_temperature()
6
7     if len(output_list) < 7:
8         output_list.append(temperature_today)
9     else:
10        output_list.pop(0)
11        output_list.append(temperature_today)
12
13    format_string = str(output_list)[1:-1]
14    print(f"The latest 7 temperatures: {format_string}.")

```

Listing 1: Python program on Lists created by a student in the control (direct instruction) group

5.1.2 Learning with Productive Failure. Requiring students in the experimental group ($n = 7$) to tackle the initial practice task on weather station data without adequate knowledge of lists elicited learning behaviours consistent with the expectations described by Kapur et al. [14]. In the students’ problem-solving efforts, there was evidence for the *activation of prior knowledge*. This included initial ideas on the list concept and the generation of representations and solution methods using other programming concepts previously learned in class. Listing 2, for example, highlights an instance where the programming task prompted the recollection of knowledge about list-like linear data structures that the student may have acquired in the past. Although the student was ultimately unable to accomplish the task, their attempted solution suggests that PF could help reinforce and reactivate their understanding of concepts they may have previously encountered but not fully grasped. Indeed, students drawing on their relevant prior skills was a common approach observed across all seven programs, with techniques involving string handling, tuples and temporary variables employed in the solution generation process.

These behaviours also shed light on another key affordance of a PF learning strategy, which is the opportunity to explore *multiple solution pathways* that may diverge from the intended canonical solution. This aligns with multiple conceptions theory proposed by Margulieux et al., in which the comparison of alternative solutions is a key factor for developing robust conceptual knowledge [16]. While no student succeeded in developing the optimal solution for the task, we identified two programs that achieved the correct functionality and satisfied all our test cases. One of these programs relied on string concatenation operations, such as `temperature = temperature + ', ' + temperature_today`, to implement the same behaviour as the `append` list method.

Static analysis of the five other solutions revealed that failure typically occurred in instances requiring knowledge of the canonical methodology. Common errors included incorrect logical statements involving variable scoping, data size checks, and data entry and removal. One such error can be seen in Line 13 of Listing 2, where the student tries to remove a data item by assigning its value to a secondary list. The errors we observed align with Kapur et al.’s guidelines for “desirable failure” [13], as students demonstrated an underlying conceptual awareness of the problem-solving task during their generation and exploration phase. This was particularly important for enabling an effective consolidation and knowledge assembly phase, in which students could reflect on their approach and develop a robust understanding of the canonical list-based solution. The outcomes of this learning process became evident in the subsequent heart-rate-based programming task, where all students optimally applied lists to construct their solutions.

```

1 from weather_station import *
2
3 temperature_list = []
4 discarded_list = []
5 temperature_count = 0
6 while NEW_DAY_AVAILABLE: # This simulates a new day
7     temperature_today = get_daily_temperature()
8
9     temperature_list += [temperature_today]
10    temperature_count += 1
11
12    if temperature_count > 7:
13        discarded_list += [temperature_list[0]]
14        temperature_count = 0
15
16    print(temperature_list)

```

Listing 2: Python program on Lists created by a student in the experimental (productive failure) group

5.1.3 Learning Outcomes. Our findings indicate that in scenarios incorporating both an instructional unit and a problem-solving task, students produced favourable outcomes regardless of the learning strategy employed. However, the results that emerged two weeks later, when students revisited the heart-rate programming task in the post-test, suggested that these effects may have been short-lived in the control group (DI). A total of three out of nine students failed to reproduce the expected solution for the task. Among them were two students who had previously performed successfully and one who had consistently failed since their first task two weeks earlier. The performance of this group raises the possibility that direct instruction is prone to “unproductive success”, where the illusion of

near transfer of knowledge may appear without authentic learning processes taking place. The views shared by students also echoed concerns about poor knowledge retention, stating they had “*easily forgotten despite learning it a mere 2 weeks ago*” and that it “*was a struggle to remember the formalities of setting up a list*”.

All students in the experimental group (PF) showed greater persistence in maintaining their initial success on the heart-rate data problem when faced with the same task two weeks later. By grappling with novel problem-solving tasks before receiving instruction, students may be more likely to internalise concepts and apply them effectively in the future.

5.2 Student Physiology

Figure 3 presents the average changes in the root mean square of successive differences between normal heartbeats (RMSSD) observed among students under different instructional conditions. Since RMSSD and cognitive load are negatively correlated, we illustrate the observed changes as $-RMSSD$ for better readability (an increase in $-RMSSD$ indicates increased cognitive load).

We observed a general trend of decreased cognitive load as students in both DI and PF groups progressed from the practice (weather) task to the programming (heart-rate) task. Students in the experimental (PF) group appeared to experience an increased cognitive load during their practice task using the weather data. Since this task was introduced before instruction in the PF group, it likely demanded a greater mental effort. On the heart-rate programming task, students in the PF and DI groups exhibited a similar change in their cognitive load overall, although students in the PF group had a much larger reduction in load when compared to their pre-instruction practice (weather) task.

Our findings mirror the core principles of Cognitive Load Theory [32], which emphasise the role of well-structured instruction in reducing cognitive load to facilitate successful learning. The lowered cognitive load induced by DI and PF in the respective tasks after instruction likely contributed to the learning outcomes discussed in Section 5.1.3. Notably, students who received instruction after PF had the highest decline in cognitive load and consequently demonstrated the best overall learning performance. As hypothesised in Kapur et al.’s theory of Productive Failure, appropriately subjecting students to an initially heavy cognitive load may be fruitful for learning [12].

5.3 Student Perceptions

We received a variety of responses to *PSQ1* and *PSQ2*, which asked students to reflect on their experiences learning Python and their preferred strategy between direct instruction and productive failure. Despite indications supporting more robust learning with PF, sentiment was more or less evenly split between the approaches, with 9 of the 16 students choosing PF as their favoured strategy. Three central themes emerged regarding the considerations for choosing the preferred learning method, which were common across responses from all students. We use the terms *effort*, *feedback* and *enjoyment* to describe these three themes. Although these themes were common to all students, their perspectives on learning effort, feedback, and enjoyment varied based on their choice of learning strategy, which we expand on below.

5.3.1 Effort to Learn. Students who preferred direct instruction commented on the efficiency of the approach. They appreciated being “*taught everything before I started work on the question*”, which they felt reduced their effort and helped them “*reach the objective faster*”. References to tags such as ‘fast’, ‘quick’, and ‘short-time’ were popular among the students, appearing in six of the nine responses. Some students offered criticisms regarding the effort required to learn using productive failure, remarking that they “*doubt I would spot [errors] in a similar speed if I were to encounter this issue in the [PF] session*” and that it would be “*challenging to solve the activities without knowing the direct option to solve it*”.

In contrast, those who preferred productive failure tended to welcome the effort needed to overcome learning challenges when expressing their thoughts about this strategy. Students viewed the difficulties encountered during the problem-solving process as beneficial learning experiences, stating that “*having to work through my memory and try and fail was a much more effective method, as it helped to ingrain the syntax and concepts in my mind better*”. The freedom to explore different ideas and solution representations encouraged students to “*experiment*” and provided “*a starting point for [me] to think from and try to come up with a solution*”.

5.3.2 Learning from Feedback. Students preferring direct instruction reported that the role of instructors and educational resources in providing feedback were key factors for their choice of learning method. Their limited knowledge of Python at the time may have prompted them to frequently rely on the instructor’s assistance during the learning phase. Representative comments include, “[*instructor_name*] provided us with information on [Python] dictionaries which I was then able to apply to efficiently” and “*I was able to get through questions pretty easily with the help of [instructor_name] supporting me*”.

Students who preferred productive failure valued the feedback they received from the mistakes made during their programming tasks. The “*firsthand experience [of] doing the programming first and slowly realising the mistake made during the coding*” cultivated learning processes centred around students iteratively generating and reflecting on their failures.

5.3.3 Enjoyment of Learning. All students appeared to enjoy the experience of both learning strategies introduced in the study. Those preferring direct instruction described their experience of learning Python programming as “*cool*”, “*great*”, and “*interesting*” (6 comments). Since the concepts taught were a novelty to most students, there was much satisfaction in getting to create “*complete*” and “*functional*” programs that explored “*how [Python] lists could be used*”. The belief that they could succeed in their programming tasks inspired confidence among the students, which may have led to increased enjoyment. For example, when reflecting on why DI was a better learning strategy for them, one student notes that it “*improve[d] my confidence ... even though it was a short activity*”.

Students preferring productive failure found the highly engaging nature of their learning experience particularly enjoyable. The absence of immediate instructor intervention in this strategy allowed students to “*feel like I came up with the solutions myself*”. This sense of ownership over their solutions motivated students to continue exploring the concepts learnt beyond the classroom, as they felt they were “*more likely to know how to apply it in the future*”. Considering the relationship between motivation and learning, using a productive failure-based strategy could stimulate intrinsic motivation, making “*the session feel more rewarding*” and “*memorable*”.

6 Conclusion

Productive failure is a learning approach that has gained popularity in STEM subjects, but its effectiveness in programming education remains under-explored. In this study, we compared Productive Failure (PF) with Direct Instruction (DI) for teaching novices about Python lists. Our key findings revealed that while initial learning outcomes were similar between the PF and DI groups, students in the PF group exhibited better knowledge retention on a post-test task delayed by two weeks. Additionally, physiological data indicated a larger reduction in cognitive load for PF students when programming after they had received instruction. Although these results are promising, the nature of our data collection limited the number of participants, and future work should look to run controlled studies at a much larger scale. In addition, collecting higher fidelity measures (such as with EEG) could provide clearer insights into factors such as cognitive load. Future work should also explore new PF activities that target different programming concepts.

Acknowledgements

This work was supported by the Assistive Augmentation research grant under the Entrepreneurial Universities (EU) initiative of New Zealand and the Tier 1 Ministry of Education (MOE) Academic Research Fund (AcRF).

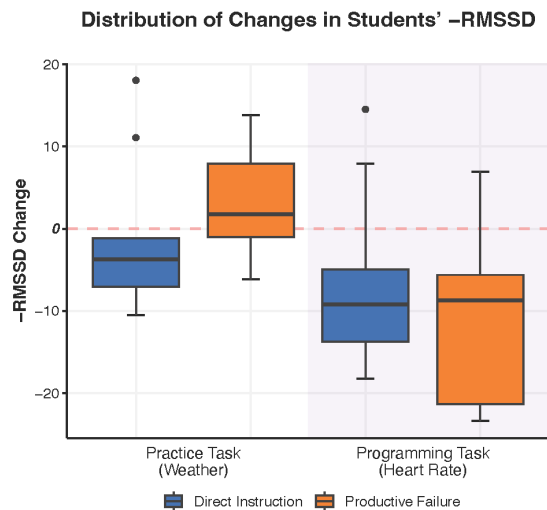


Figure 3: Distribution of changes in students' RMSSD during the practice (weather) task and the programming (heart-rate) task. A -RMSSD change that is less than 0 correlates to lowered cognitive load.

References

- [1] Olivier Augereau, Kai Kunze, and Koichi Kise. 2019. Experimental Supplements from Mobile Tools for Cognitive Introspection Towards Cognitive Augmentation. *GetMobile: Mobile Comp. and Comm.* 23, 2 (Nov. 2019), 22–24. <https://doi.org/10.1145/3372300.3372305>
- [2] David C. Berliner. 2002. Comment: Educational Research: The Hardest Science of All. *Educational Researcher* 31, 8 (2002), 18–20. <https://doi.org/10.3102/0013189X031008018>
- [3] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101. <https://doi.org/10.1191/1478088706qp0630a>
- [4] Ibrahim Dahlstrom-Hakki, Jodi Asbell-Clarke, and Elizabeth Rowe. 2019. Showing is knowing: The potential and challenges of using neurocognitive measures of implicit learning in the classroom. *Mind, Brain, and Education* 13, 1 (2019), 30–40. <https://doi.org/10.1111/mbe.12177>
- [5] Paul Denny, Brett A. Becker, Michelle Craig, Greg Wilson, and Piotr Banaszekiewicz. 2019. Research This! Questions that Computing Educators Most Want Computing Education Researchers to Answer. In *Proceedings of the 2019 ACM Conference on International Computing Education Research* (Toronto ON, Canada) (ICER '19). Association for Computing Machinery, New York, NY, USA, 259–267. <https://doi.org/10.1145/3291279.3339402>
- [6] Juan Carlos Farah, Arielle Moro, Kristoffer Bergram, Aditya Kumar Purohit, Denis Gillet, and Adrian Holzer. 2020. Bringing computational thinking to non-STEM undergraduates through an integrated notebook application. In *15th European Conference on Technology Enhanced Learning*.
- [7] Eija Haapalainen, SeungJun Kim, Jodi F. Forlizzi, and Anind K. Dey. 2010. Psychophysiological measures for assessing cognitive load. In *Proceedings of the 12th ACM International Conference on Ubiquitous Computing* (Copenhagen, Denmark) (UbiComp '10). Association for Computing Machinery, New York, NY, USA, 301–310. <https://doi.org/10.1145/1864349.1864395>
- [8] Idit Ed Harel and Seymour Ed Papert. 1991. *Constructionism*. Ablex Publishing.
- [9] Association for Computing Machinery (ACM) Joint Task Force on Computing Curricula and IEEE Computer Society. 2013. *Computer Science Curricula 2013: Curriculum Guidelines for Undergraduate Degree Programs in Computer Science*. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2534860>
- [10] Boonserm Kaewkamnerdpong. 2016. A Framework for Human Learning Ability Study Using Simultaneous EEG/fNIRS and Portable EEG for Learning and Teaching Development. In *Smart Education and e-Learning 2016*, Vladimir L. Uskov, Robert J. Howlett, and Lakshmi C. Jain (Eds.). Springer International Publishing, Cham, 155–165. https://doi.org/10.1007/978-3-319-39690-3_14
- [11] Manu Kapur. 2008. Productive Failure. *Cognition and Instruction* 26, 3 (2008), 379–424. <https://doi.org/10.1080/07370000802212669>
- [12] Manu Kapur. 2016. Examining Productive Failure, Productive Success, Unproductive Failure, and Unproductive Success in Learning. *Educational Psychologist* 51, 2 (2016), 289–299. <https://doi.org/10.1080/00461520.2016.1155457>
- [13] Manu Kapur and Katherine Bielaczyc. 2012. Designing for Productive Failure. *Journal of the Learning Sciences* 21, 1 (2012), 45–83. <https://doi.org/10.1080/1058406.2011.591717>
- [14] Manu Kapur and Nikol Rummel. 2012. Productive failure in learning from generation and invention activities. *Instructional Science* 40 (2012), 645–650. <https://doi.org/10.1007/s11251-012-9235-4>
- [15] Katharina Loibl, Ido Roll, and Nikol Rummel. 2017. Towards a theory of when and how problem solving followed by instruction supports learning. *Educational psychology review* 29 (2017), 693–715. <https://doi.org/10.1007/s10648-016-9379-x>
- [16] Lauren Margulieux, Paul Denny, Kathryn Cunningham, Michael Deutsch, and Benjamin R. Shapiro. 2021. When Wrong is Right: The Instructional Power of Multiple Conceptions. In *Proceedings of the 17th ACM Conference on International Computing Education Research* (Virtual Event, USA) (ICER 2021). Association for Computing Machinery, New York, NY, USA, 184–197. <https://doi.org/10.1145/3446871.3469750>
- [17] Caitlin Mills, Igor Fridman, Walid Soussou, Disha Waghay, Andrew M. Olney, and Sidney K. D'Mello. 2017. Put your thinking cap on: detecting cognitive load using EEG during learning. In *Proceedings of the Seventh International Learning Analytics & Knowledge Conference* (Vancouver, British Columbia, Canada) (LAK '17). Association for Computing Machinery, New York, NY, USA, 80–89. <https://doi.org/10.1145/3027385.3027431>
- [18] Reinhard Pekrun and Markus Böhner. 2014. Self-report Measures of Academic Emotions. *International Handbook of Emotions in Education* (2014), 561–579.
- [19] W James Popham. 2001. *The truth about testing: An educator's call to action*. ASCD.
- [20] Lauren B. Resnick and Daniel P. Resnick. 1992. Assessing the Thinking Curriculum: New Tools for Educational Reform. In *Changing Assessments: Alternative Views of Aptitude, Achievement and Instruction*, Bernard R. Gifford and Mary Catherine O'Connor (Eds.). Springer Netherlands, Dordrecht, 37–75. https://doi.org/10.1007/978-94-011-2968-8_3
- [21] Philip Sands. 2019. Addressing cognitive load in the computer science classroom. *ACM Inroads* 10, 1 (Feb. 2019), 44–51. <https://doi.org/10.1145/3210577>
- [22] Zachary Monroe Savelson. 2020. *Student Emotions in a Productive Failure Paradigm*. Ph.D. Dissertation. Carleton University.
- [23] Zachary M. Savelson and Kasia Muldner. 2023. How do students feel and collaborate during programming activities in the productive failure paradigm? *Computer Science Education* 0, 0 (2023), 1–34. <https://doi.org/10.1080/08993408.2023.2237365> arXiv:https://doi.org/10.1080/08993408.2023.2237365
- [24] Lennart Schalk, Ralph Schumacher, Armin Barth, and Elsbeth Stern. 2018. When problem-solving followed by instruction is superior to the traditional tell-and-practice sequence. *Journal of educational psychology* 110, 4 (2018), 596.
- [25] Fred Shaffer and Jay P Ginsberg. 2017. An overview of heart rate variability metrics and norms. *Frontiers in public health* 5 (2017), 258. <https://doi.org/10.3389/fpubh.2017.00258>
- [26] Tanmay Sinha and Manu Kapur. 2021. When problem solving followed by instruction works: Evidence for productive failure. *Review of Educational Research* 91, 5 (2021), 761–798. <https://doi.org/10.3102/00346543211019105>
- [27] Priyashri K. Sridhar, Samantha W.T. Chan, Yvonne Chua, Yow Wei Quin, and Suranga Nanayakkara. 2019. Going beyond performance scores: Understanding cognitive-affective states in Kindergarteners and application of framework in classrooms. *International Journal of Child-Computer Interaction* 21 (2019), 37–53. <https://doi.org/10.1016/j.ijcci.2019.04.002>
- [28] Priyashri Kamlesh Sridhar and Suranga Nanayakkara. 2020. Progression of Cognitive-Affective States During Learning in Kindergarteners: Bringing Together Physiological, Observational and Performance Data. *Education Sciences* 10 (2020), 177. <https://doi.org/10.3390/educsci10070177>
- [29] Phil Steinhorst, Christof Duhme, Xiaoyi Jiang, and Jan Vahrenhold. 2024. Recognizing Patterns in Productive Failure. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1* (Portland, OR, USA) (SIGCSE 2024). Association for Computing Machinery, New York, NY, USA, 1293–1299. <https://doi.org/10.1145/3626252.3630915>
- [30] Phil Steinhorst, Andrew Petersen, Bogdan Simion, and Jan Vahrenhold. 2023. Exploring Barriers in Productive Failure. In *Proceedings of the 2023 ACM Conference on International Computing Education Research - Volume 1* (Chicago, IL, USA) (ICER '23). Association for Computing Machinery, New York, NY, USA, 284–297. <https://doi.org/10.1145/3568813.3600111>
- [31] Hussel Suriyaarachchi, Paul Denny, Juan Pablo Forero Cortes, Chamod Weerasinghe, and Suranga Nanayakkara. 2022. Primary School Students Programming with Real-Time Environmental Sensor Data. In *Proceedings of the 24th Australasian Computing Education Conference* (Virtual Event, Australia) (ACE '22). Association for Computing Machinery, New York, NY, USA, 85–94. <https://doi.org/10.1145/3511861.3511871>
- [32] John Sweller. 1994. Cognitive load theory, learning difficulty, and instructional design. *Learning and Instruction* 4, 4 (1994), 295–312. [https://doi.org/10.1016/0959-4752\(94\)90003-5](https://doi.org/10.1016/0959-4752(94)90003-5)
- [33] Aik Lim Tan, Robyn Gillies, and Azilawati Jamaludin. 2021. A Case Study: Using a Neuro-Physiological Measure to Monitor Students' Interest and Learning during a Micro-Bit Activity. *Education Sciences* 11 (2021), 379. <https://doi.org/10.3390/educsci11080379>
- [34] Julian F Thayer, Anita L Hansen, Evelyn Saus-Rose, and Bjorn Helge Johnsen. 2009. Heart rate variability, prefrontal neural function, and cognitive performance: the neurovisceral integration perspective on self-regulation, adaptation, and health. *Annals of behavioral medicine* 37, 2 (2009), 141–153. <https://doi.org/10.1007/s12160-009-9101-z>
- [35] Sverrir Thorgeirsson, Tanmay Sinha, Felix Friedrich, and Zhendong Su. 2022. Does Deliberately Failing Improve Learning Introductory Computer Science?. In *Educating for a New Future: Making Sense of Technology-Enhanced Learning Adoption: 17th European Conference on Technology Enhanced Learning, EC-TEL 2022, Toulouse, France, September 12–16, 2022, Proceedings* (Toulouse, France). Springer-Verlag, Berlin, Heidelberg, 608–614. https://doi.org/10.1007/978-3-031-16290-9_57
- [36] Sverrir Thorgeirsson and Zhendong Su. 2021. Algot: An Educational Programming Language with Human-Intuitive Visual Syntax. In *2021 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)* (St Louis, MO, USA). IEEE, 1–5. <https://doi.org/10.1109/VL/HCC51201.2021.9576166>
- [37] Rebecca J. Trueman. 2014. Productive Failure in Stem Education. *Journal of Educational Technology Systems* 42, 3 (2014), 199–214. <https://doi.org/10.2190/ET.42.3.b>
- [38] Paul van Gent, Haneen Farah, Nicole van Nes, and Bart van Arem. 2019. HeartPy: A novel heart rate algorithm for the analysis of noisy signals. *Transportation Research Part F: Traffic Psychology and Behaviour* 66 (2019), 368–378. <https://doi.org/10.1016/j.trf.2019.09.015>